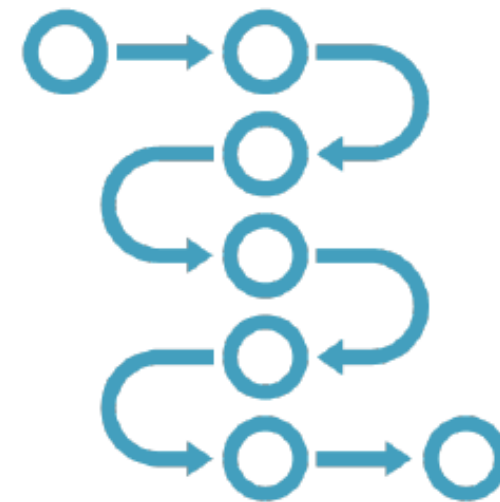




自定义异常与异常处理链

北京理工大学计算机学院
金旭亮

异常处理策略概述

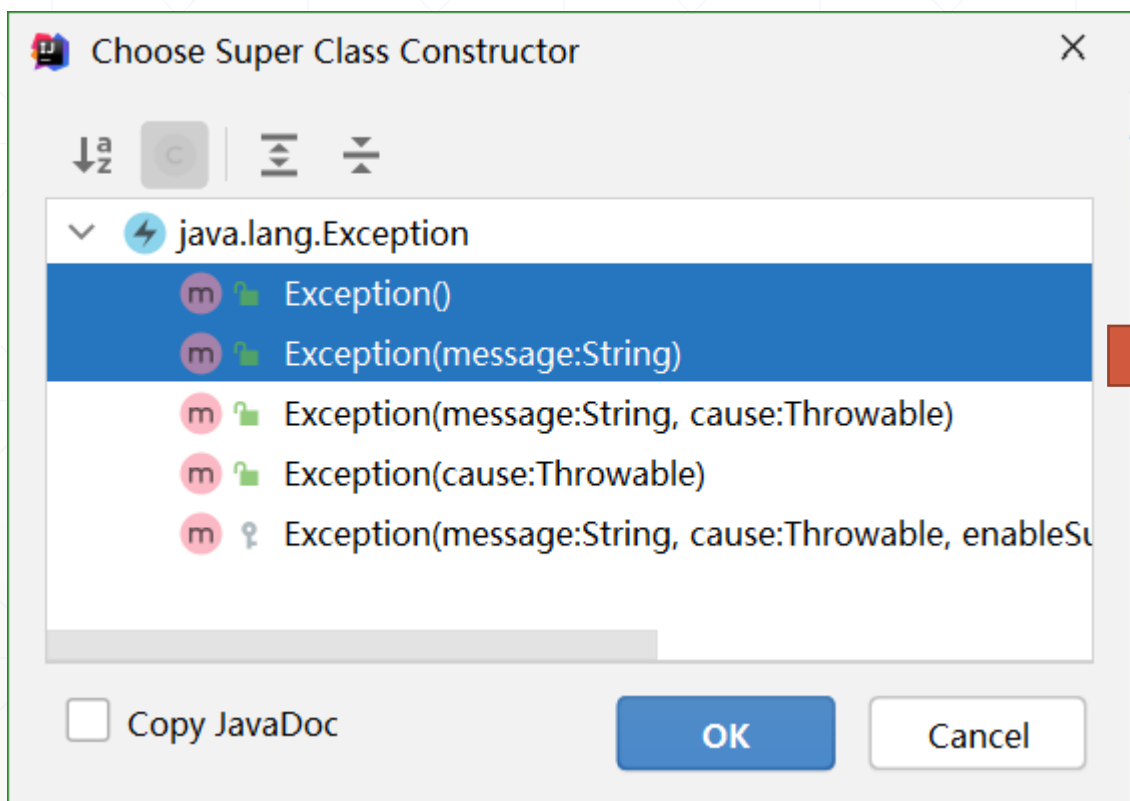
- 在实际开发中，选择正确的异常处理策略非常重要，它是系统健壮性的重要保证。



- 在长期的软件开发实践中，人们已经总结出了多种切实可行的异常处理方法和策略，在本讲中，我们介绍一种被广泛使用的异常处理方法——通过自定义异常类捕获并处理业务逻辑错误。

创建自己的异常类

自定义异常通常选择直接派生自Exception或RuntimeException:



//自定义异常类，通常需要重写基类的构造方法

```
public class MyException extends Exception {  
    public MyException() {  
    }  
  
    public MyException(String message) {  
        super(message);  
    }  
}
```

派生自Exception，就定义了一个受控异常，抛出之后，要求使用者必须捕获或再次抛出。

抛出自定义异常类对象

在合适的地方使用throw语句抛出自定义异常对象：

```
class MyClass {  
    //检测某些条件是否满足  
    boolean testSomeCondition() {  
        //...  
        return true;  
    }  
  
    void someMethod() throws MyException {  
        //当某些条件被满足时,抛出自定义的异常  
        if (testSomeCondition() == true) {  
            throw new MyException();  
        }  
        //其它代码  
    }  
}
```

自定义异常通常与业务逻辑密切相关，比如尝试删除一条不存在的订单时，就可以选择抛出一个自定义的异常（并携带着必要的业务数据或信息），以向外部表明出现了这种违背业务逻辑约束条件的情况。

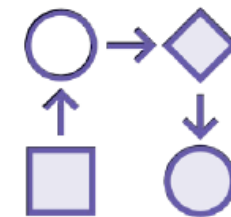
外界捕获这种异常，通过异常的具体类型，以及异常对象携带的信息，就可以决定应对策略了。

捕获自定义异常类

```
public class UseMyException {  
    public static void main(String[] args) {  
        var obj = new MyClass();  
        try {  
            obj.someMethod();  
        } catch (MyException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

可以在catch块中取出异常对象携带的信息，进行特定的处理工作

构建业务逻辑异常处理链



- 在实际开发中，经常需要将特定的“过于专业的异常（比如JDK中定义的许多异常，它们对于普通用户来说是没什么意义的，也是难与理解的）”转换为一个“业务逻辑类型”的异常（这种异常，普通用户是可以理解的，因为它只与具体业务相关，而与特定的底层软件技术细节无关。），然后在调用者处进行捕获与处理。
- 为完成这个任务，通常会在程序中构建出一条“业务逻辑异常处理链”。
- `ExceptionLinkInRealWorld.java` 示例展示了典型的“业务逻辑异常处理链”编程套路。

自定义业务逻辑异常类

```
//自定义的异常类,通常用于代表特定的业务逻辑层的异常
//表明在处理业务过程中出现了某种“意外”
class BusinessException extends Exception {
    public BusinessException(String Message) {
        super(Message);
    }
    public BusinessException(String message, Throwable cause) {
        super(message, cause);
    }
    public BusinessException(Throwable cause) {
        super(cause);
    }
    //可以添加其他字段,以表明特定的业务逻辑相关的信息
}
```

如果此异常对应的业务非常重要,必须及时处理,就定义“受控异常”,否则,可以定义“运行时异常”

为了方便程序员了解异常的底层技术细节,可以选择将原始异常对象“保存”起来,以便后继进行分析或跟踪

为了方便外界处理,业务逻辑类型的异常,应该携带有足够的附加信息(比如订单Id,创建时间之类)

实现业务逻辑的方法示例

```
//某个实现业务逻辑的方法，当检测到出现非法状态时，  
//可以主动抛出一个“业务逻辑”异常  
public static void someBusinessMethod()  
    throws MyBusinessException {  
    try {  
        System.out.println("Method throwException");  
        if(somethingIsWrong)  
            throw new MyBusinessException("somethingIsWrong");  
    } catch (Exception e) {  
        //JDK引发的，或者其它特定的异常，可以根据实际情况，  
        //选择一个异常处理策略：  
        // (1) 直接本地处理  
        // (2) 不处理，直接抛出，让方法的调用者处理  
        // (3) 将它转换为一个自定义业务逻辑异常，再抛出  
        throw new MyBusinessException("做XXX事情时出错了!", e);  
    } finally {  
        //在此完成一些收尾处理工作，比如写日志  
    }  
}
```

无需抛出业务异常对象的普通方法的示例

```
//普通的, 无需特殊处理的方法
public static void doesNotThrowException() {
    try {
        System.out.println("Method doesNotThrowException");
    } catch (Exception e) {
        System.err.println(e.toString());
    } finally {
        System.err.println(
            "Finally executed in doesNotThrowException");
    }
    System.out.println(
        "End of method doesNotThrowException");
}
```



这种类型的方法，通常在其内部封装有特定的异常处理逻辑，在“本地解决”所有异常情况。

实现业务处理流程

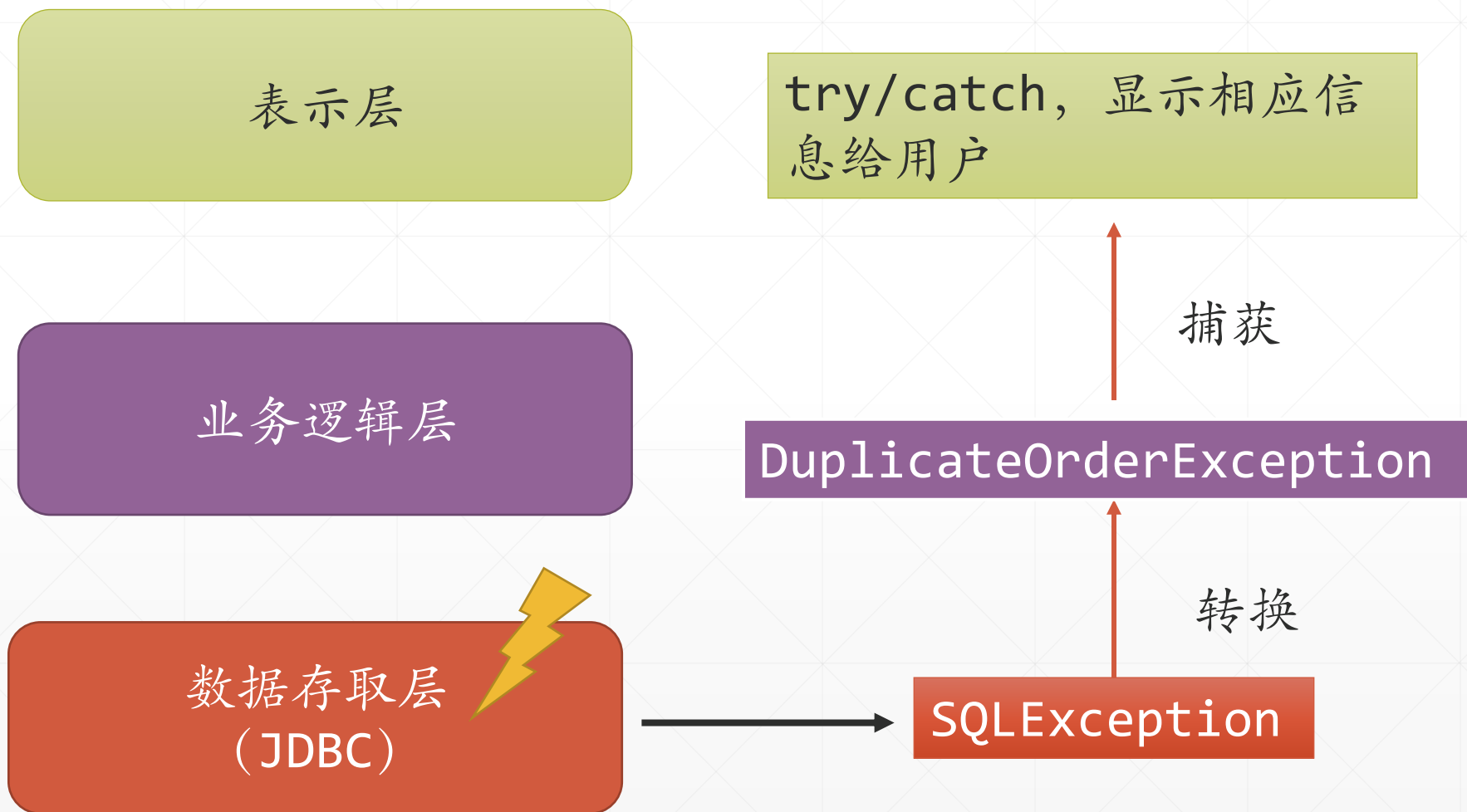
在实际项目中，一个业务流程通常会包容多个具体步骤，每个步骤可用一个业务逻辑方法实现，并且可以抛出不同类型的自定义业务逻辑异常，外界可以有针对性地捕获这些异常对象，或再次抛出它们。

```
public static void main(String args[]) {  
    try {  
        someBusinessMethod();  
        doesNotThrowException();  
        //可以按照特定流程的步骤，在此集成多个方法  
        //.....  
    } catch (MyBusinessException e) {  
        //处理业务逻辑相关的异常  
        //如果有多个，就写多个catch块  
    } catch (Exception e) {  
        //处理所有未捕获的异常  
    }  
}
```



在实际开发中，可以参照ExceptionLinkInRealWorld.java示例的作法，定义一些与业务逻辑相关的自定义异常类，供上层代码进行捕获，从而能更精确地反映系统真实运行情况并及时进行处理。

一个电商网站的异常处理流程



小结：关于开发中异常处理的建议



- 在中间层组件中抛出异常，在界面层组件中捕获异常
- 在底层组件中捕获JVM抛出的“只有程序员能看懂的”异常，转换为中间层的业务逻辑异常，再由界面层捕获以提供有意义的信息。
- 自身能够处理的异常，不要再向外界抛出。
- 尽可能地在靠近异常发生的地方捕获并处理异常。
- 尽可能地捕获最具体的异常类型，不要在中间层用catch(Exception)“吃掉”所有异常
- 在开发阶段捕获并显示所有异常信息，发布阶段要移除部分代码，以避免“过于专业”的异常信息困扰用户，特别地，系统发布之后，不要将服务端异常的详细信息发给客户端，以免被黑客利用。

动手动脑



- 编写一个程序，此程序在运行时要求用户输入一个整数，代表某门课的考试成绩，程序接着给出“不及格”、“及格”、“中”、“良”、“优”的结论。
- 要求程序必须具备足够的健壮性，不管用户输入什么样的内容，都不会崩溃。

动手动脑



大家在访问网站时，司空见惯的“用户登录”功能，其实是包容着一系列特定步骤的业务处理流程，比如，用户在网页或App中输入用户名与密码之后，首先就要保证它们符合要求（比如用户名不能为空，密码不能少于6位之类），之后，这些信息被发往数据库，提取用户信息进行比对.....

请从“业务逻辑”的角度，分析一下“用户登录”流程，并为它设计出相应的自定义业务逻辑异常，写一个小的Demo，构建出你的“用户登录”处理流程。